**Universidad Mayor de San Andrés**

Facultad de Ciencias Puras y Naturales

Carrera de Informática

# Sistema multiplataforma de inventario para una tienda con base de datos en la nube

*Inventario Cloud*

**Proyecto de la materia**

**Materia:** Computación en la Nube

**Estudiantes:** Jorge Alberto Ichuta Tuco

Alex Ronaldo Huayllucu Huayllucu

José Fernando Ignacio Mamani

David Jhon Mamani Vallejo

Cesar Raymundo Claros Choquemisa

**Docente:** Ph. D. José María Tapia Baltazar

La Paz – Bolivia, octubre de 2026

# 1 Introducción

En casi cualquier tienda es importante saber cuántos productos hay en el almacén. De eso depende cuándo hacer un pedido al proveedor, qué se le puede vender a un cliente y cuánto dinero hay invertido en mercadería. Sin embargo, en muchas tiendas de barrio y minimarkets de La Paz y El Alto el inventario todavía se lleva en un cuaderno o en una hoja de Excel que está en una sola computadora.

Mientras la tienda es pequeña esto funciona, pero con el tiempo aparecen los problemas: lo anotado no coincide con lo que hay en los estantes, se repiten registros, se pierden hojas y nadie sabe bien qué productos se están acabando. Además, si el dueño no está en la tienda, no tiene forma de ver cómo va el inventario.

En la materia de Computación en la Nube vimos que hoy se pueden usar servicios de internet para guardar datos y hacer funcionar aplicaciones sin comprar un servidor propio. Con esa idea hicimos *Inventario Cloud*, un sistema de inventario para una tienda que se usa como aplicación web: se abre desde el navegador del celular o de la computadora, sin instalar nada.

La aplicación está hecha con Flutter y guarda los datos en una base de datos PostgreSQL que está en Supabase. Todos los dispositivos usan la misma base de datos, así que si un empleado registra una venta desde su celular, el administrador ve el stock actualizado en su computadora. Para todo el proyecto usamos solo planes gratuitos, por lo que el costo fue de Bs 0.

En este documento primero explicamos los conceptos que usamos (sección 2), luego describimos el problema (sección 3), los objetivos (sección 4) y al final los servicios que usa y ofrece el sistema (sección 5).

# 2 Marco teórico

## 2.1 Control de inventario

Un inventario es el conjunto de productos que una empresa tiene guardados para vender [4]. Llevar el control del inventario consiste en anotar todo lo que entra (compras a proveedores) y todo lo que sale (ventas o productos dañados), para saber en cualquier momento cuánto queda de cada producto. A esa cantidad se le llama *stock* o existencia.

En nuestro sistema cada producto tiene además un stock mínimo. Cuando el stock baja hasta ese número, el sistema muestra una alerta para que el encargado sepa que tiene que volver a comprar. También se guarda un historial de movimientos: cada vez que el stock sube o baja queda registrado quién lo hizo, cuándo y por qué. Si en un conteo físico se encuentra una diferencia, no se borra nada; se hace un ajuste que

también queda en el historial.

## 2.2 Computación en la nube

El NIST (Instituto Nacional de Estándares y Tecnología de Estados Unidos) define la computación en la nube como un modelo que permite usar, a través de internet y cuando uno lo necesite, recursos como servidores, almacenamiento y aplicaciones, sin tener que administrarlos directamente [6]. Según esta definición, la nube tiene cinco características: el usuario puede pedir recursos por su cuenta, se accede desde cualquier dispositivo con internet, los recursos se comparten entre muchos clientes, se pueden aumentar o reducir según la necesidad, y el uso se mide para cobrarlo o limitarlo.

Para una tienda pequeña la ventaja es clara: en lugar de comprar un servidor, se paga solo por lo que se usa, y en proyectos pequeños muchas veces ni siquiera se paga porque alcanza con el plan gratuito [1].

Los servicios en la nube se suelen dividir en tres modelos. En IaaS (infraestructura como servicio) el proveedor da máquinas virtuales y el cliente instala todo lo demás, por ejemplo Amazon EC2. En PaaS (plataforma como servicio) el proveedor da un entorno listo para publicar aplicaciones, como Netlify. En SaaS (software como servicio) se usa una aplicación ya terminada, como Gmail o GitHub. Hay un cuarto tipo que usamos en este proyecto, el BaaS (backend como servicio), donde el proveedor ya da la base de datos, el inicio de sesión y la API, y el desarrollador solo tiene que hacer la parte visual.

También se habla de nube pública, privada, comunitaria e híbrida [6]. Nosotros usamos una nube pública, porque los servicios de Supabase y Netlify están abiertos a cualquier persona por internet.

## 2.3 Supabase

Supabase es una plataforma de código abierto del tipo BaaS que funciona sobre PostgreSQL [9]. Al crear un proyecto, Supabase entrega una base de datos, un servicio para registrar usuarios e iniciar sesión, y una API que se genera sola a partir de las tablas. También permite crear funciones dentro de la base de datos y llamarlas desde la aplicación. Tiene un plan gratuito que alcanza para proyectos pequeños como el nuestro [10].

## 2.4 Base de datos relacional y PostgreSQL

Una base de datos relacional guarda la información en tablas que se relacionan entre sí por medio de claves [8]. La clave primaria identifica a cada registro y la clave foránea es un campo que apunta a un registro de otra tabla. Gracias a esto no se puede,

por ejemplo, registrar la venta de un producto que no existe.

PostgreSQL es un gestor de bases de datos libre y gratuito [7]. De todo lo que ofrece, en el proyecto aprovechamos sobre todo cuatro cosas. La primera son las transacciones: una venta con varios productos se guarda completa o no se guarda nada. La segunda son las restricciones, que impiden guardar datos incorrectos, como un stock negativo o un código de producto repetido. La tercera son las funciones, que nos permiten registrar una venta, descontar el stock y guardar el movimiento en un solo paso. La cuarta es la seguridad por filas (RLS, *Row Level Security*), que decide qué datos puede ver o cambiar cada usuario según su rol.

## 2.5 API, HTTPS y JSON

Una API es el intermediario entre la aplicación y la base de datos. La aplicación no se conecta directo a la base, sino que le hace pedidos a la API por internet. Supabase usa una API de tipo REST, un estilo propuesto por Fielding [2] en el que se piden o envían datos usando los métodos de HTTP. Los datos viajan en formato JSON y por HTTPS, que cifra la información. Una ventaja de trabajar así es que la contraseña de la base de datos nunca queda guardada en los celulares ni en las computadoras.

## 2.6 Inicio de sesión y roles

Para entrar al sistema cada persona usa su correo y su contraseña. Supabase revisa esos datos y, si son correctos, entrega un token (un JWT [5]) que la aplicación envía en cada pedido. Con ese token la base de datos sabe quién está pidiendo los datos y qué puede hacer. Las contraseñas no se guardan tal cual, sino cifradas, y eso lo maneja Supabase.

En el sistema hay dos roles. El administrador puede manejar todo: productos, categorías, proveedores, entradas y usuarios. El empleado puede ver los productos, registrar ventas y ver su propio historial.

## 2.7 Flutter y aplicación web

Flutter es una herramienta gratuita de Google para hacer aplicaciones para celular, web y escritorio con un solo código, escrito en el lenguaje Dart [3]. Nosotros la usamos para crear una aplicación web que publicamos gratis en Netlify, y que se abre desde el navegador de cualquier celular o computadora.

La aplicación se acomoda al tamaño de la pantalla. En la computadora muestra un menú al costado y tablas, y en el celular una barra de menú abajo y tarjetas. A esto se le llama diseño adaptable o *responsive*.

Por último, aunque los datos estén en la nube, igual hay que sacar copias de respaldo. El plan gratuito de Supabase no las hace solo [10], así que cada cierto tiempo exportamos la base de datos y la guardamos aparte.

# 3 Problemática – Identificación del problema

## 3.1 Situación actual

En muchas tiendas pequeñas el inventario se anota en cuadernos o en hojas de cálculo. Cada empleado registra las ventas a su manera, las compras a veces se anotan tarde o se olvidan, y toda la información depende de una sola computadora o de un solo cuaderno.

Por eso el dueño no sabe con seguridad cuánto tiene de cada producto. Hay productos que se acaban sin que nadie se dé cuenta y otros que se compran de más y se quedan guardados. Revisar qué se vendió la semana pasada o qué proveedor trajo cierto producto toma mucho tiempo.

## 3.2 Identificación del problema

Después de analizar la situación, identificamos como problema principal que *la tienda no tiene un control centralizado de su inventario, y por eso la información de stock, compras y ventas no es confiable ni se puede ver desde varios lugares al mismo tiempo.*

Para entender mejor el problema hicimos un árbol de problemas (figura 1). Abajo están las causas, al centro el problema y arriba los efectos.

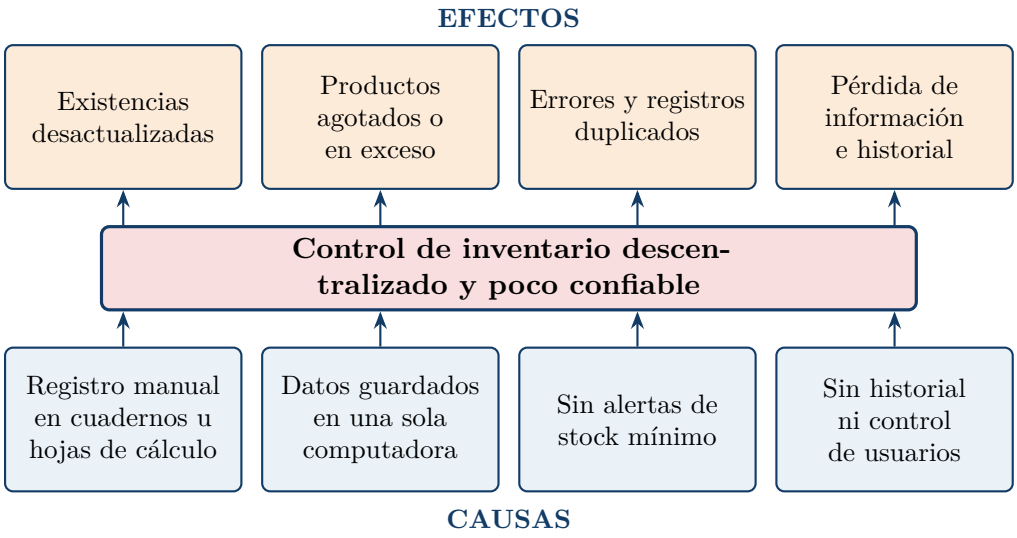



**Figura 1:** Árbol de problemas.

### 3.3 Causas y consecuencias

En el cuadro 1 juntamos cada causa con lo que provoca y con lo que hace el sistema para solucionarlo.

| Causa | Consecuencia | Qué hace el sistema |
|---|---|---|
| Se anota todo a mano | Errores, registros repetidos y stock incorrecto | Formularios que revisan los datos y actualizan el stock solos |
| Los datos están en una sola computadora | No se pueden ver desde otro lugar y se pueden perder | Una sola base de datos en la nube, que se ve desde el celular o la PC |
| No hay avisos | Productos que se acaban sin que nadie se dé cuenta | Alerta cuando el stock llega al mínimo |
| No hay historial | No se sabe cuándo ni por qué cambió el stock | Cada entrada, venta y ajuste queda guardado |
| Cualquiera puede cambiar los datos | No se sabe quién hizo cada cambio | Inicio de sesión y permisos por rol |
| No hay copias | Si algo falla, se pierde todo | Respaldos de la base de datos |

**Cuadro 1:** Causas, consecuencias y solución propuesta.

### 3.4 Formulación del problema

Con todo lo anterior, el problema se puede resumir en la siguiente pregunta: ¿cómo lograr que la información del inventario de una tienda esté en un solo lugar, siempre actualizada y que se pueda consultar de forma segura desde el celular o la computadora, sin gastar en servidores?

## 4 Objetivos

### 4.1 Objetivo general

Desarrollar un sistema de inventario para una tienda, que se pueda usar desde celulares y computadoras y que guarde sus datos en una base de datos en la nube, para registrar y consultar productos, categorías, proveedores, stock, entradas y ventas, y así tener un mejor control del inventario con menos errores.

### 4.2 Objetivos específicos

1. Diseñar una base de datos relacional para guardar los productos, categorías, proveedores, entradas, ventas y movimientos de inventario.

2. Hacer una interfaz que se pueda usar cómodamente en el celular y en la computadora.
3. Programar las funciones para registrar, consultar, modificar y desactivar los datos del sistema.
4. Registrar las entradas de mercadería y las ventas, y que el stock se actualice solo.
5. Mostrar una alerta cuando un producto llegue a su stock mínimo.
6. Guardar un historial de todos los movimientos del inventario.
7. Poner la base de datos en un servicio en la nube para poder usarla desde cualquier lugar.
8. Lograr que los cambios hechos en un dispositivo se vean en los demás.
9. Agregar seguridad básica: inicio de sesión, permisos por rol y protección de las claves.
10. Sacar respaldos de la base de datos para no perder información.
11. Hacer pruebas para comprobar que el sistema funciona bien y que los datos se guardan correctamente.

# 5 Lista de servicios y su descripción

El sistema usa varios servicios gratuitos de la nube y, a la vez, ofrece servicios a las personas que trabajan en la tienda. En la figura 2 se ve cómo se conectan: el celular y la computadora abren la aplicación en el navegador, y esta se comunica por internet con Supabase, donde están el inicio de sesión, la API y la base de datos.

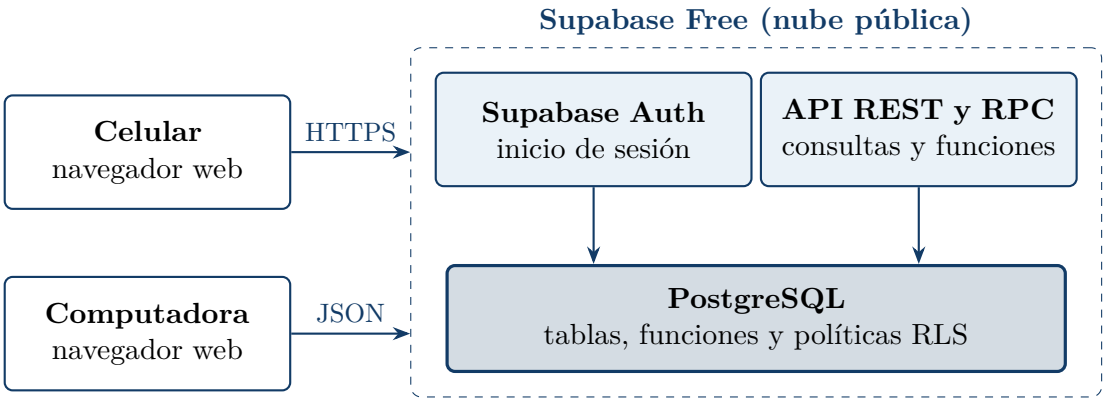



**Figura 2:** Arquitectura del sistema.

## 5.1 Servicios en la nube que usa el sistema

En el cuadro 2 están los servicios de la nube que usamos y para qué sirve cada uno.

| Servicio | Tipo | Para qué lo usamos |
|---|---|---|
| Base de datos de Supabase (PostgreSQL) | BaaS | Guarda todos los datos del sistema: usuarios, productos, categorías, proveedores, entradas, ventas y movimientos. |
| Supabase Auth | BaaS | Permite iniciar sesión con correo y contraseña. Guarda las contraseñas cifradas. |
| API de Supabase | BaaS | Es por donde la aplicación pide y envía los datos. También permite llamar a las funciones que registran entradas, ventas y ajustes. |
| Seguridad por filas (RLS) | BaaS | Reglas dentro de la base de datos que deciden qué puede ver y hacer cada rol. |
| Netlify | PaaS | Aloja la aplicación web de forma gratuita. El sistema está publicado en `https://melodic-melomakarona-8b8ae7.netlify.app`. |
| GitHub | SaaS | Guarda el código del proyecto y su historial de cambios. |

**Cuadro 2:** Servicios en la nube usados en el proyecto.

Todo el proyecto se hizo con un presupuesto de Bs 0. En octubre de 2026 el plan gratuito de Supabase da 500 MB de base de datos, 1 GB para archivos, 5 GB de transferencia al mes y hasta dos proyectos activos [10]. Para una tienda esto alcanza de sobra. Lo que sí hay que tener en cuenta es que el plan gratuito no saca respaldos automáticos y que pausa el proyecto si pasa una semana sin usarse, por eso conviene entrar al sistema al menos una vez por semana.

## 5.2 Modelo de la base de datos

Antes de crear las tablas en Supabase hicimos el modelo entidad-relación, que se muestra en la figura 3. Cada recuadro es una tabla y debajo del nombre están sus campos. PK es la clave primaria y FK es una clave foránea. En las líneas, el 1 y la N indican cuántos registros se relacionan de cada lado.

El modelo se lee así: una categoría tiene muchos productos y cada producto pertenece a una sola categoría. Un proveedor puede tener muchas entradas de mercadería. Cada entrada y cada venta tienen su detalle, donde se guarda qué productos entraron o se vendieron, cuántos y a qué precio. Un usuario puede registrar muchas entradas, ventas y movimientos. Y cada vez que cambia el stock de un producto se guarda un registro en la tabla de movimientos, con el stock que había antes y el que quedó después.

La tabla de perfiles está unida a la tabla de usuarios de Supabase (`auth.users`), que es donde se guardan los correos y las contraseñas, así que nosotros no guardamos ninguna contraseña en nuestras tablas. Otra decisión importante fue que el stock no se puede cambiar a mano: solo cambia cuando se registra una entrada, una venta o un

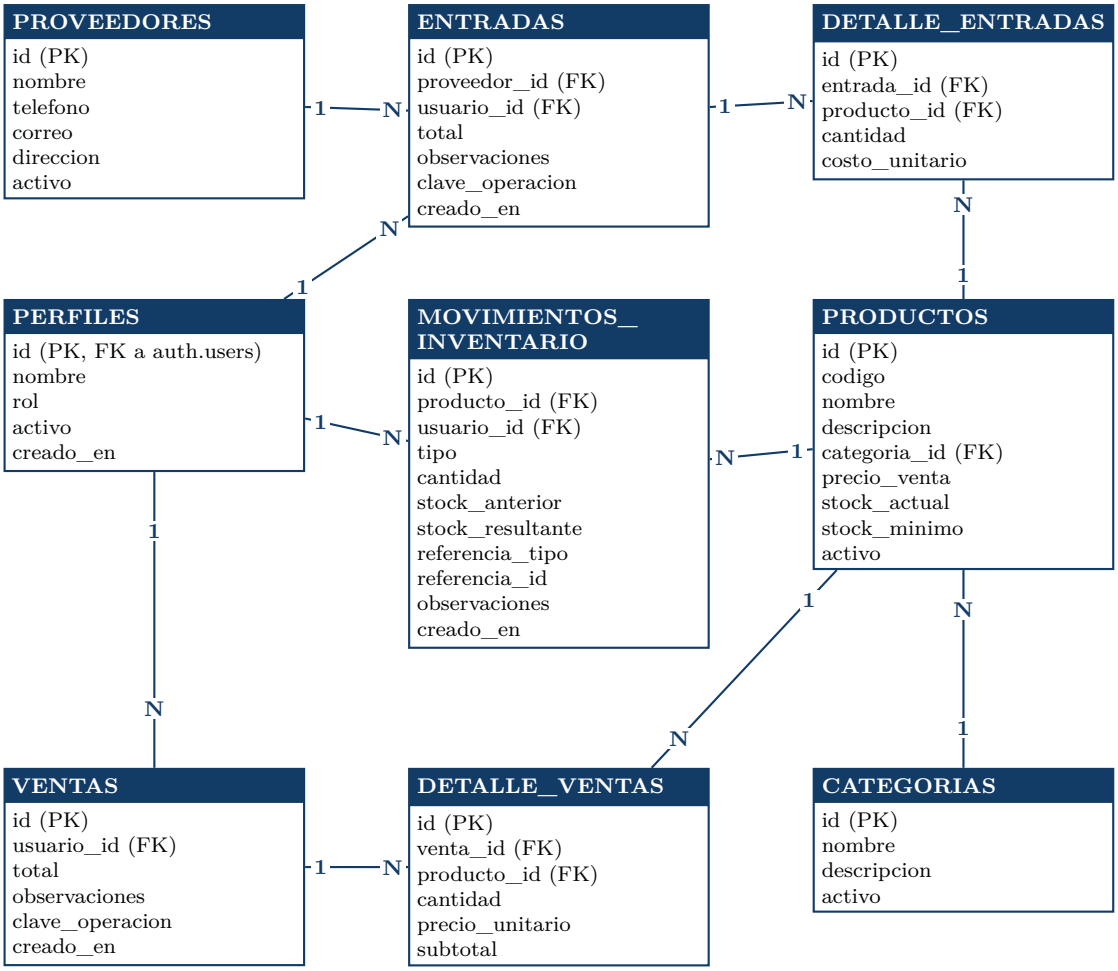
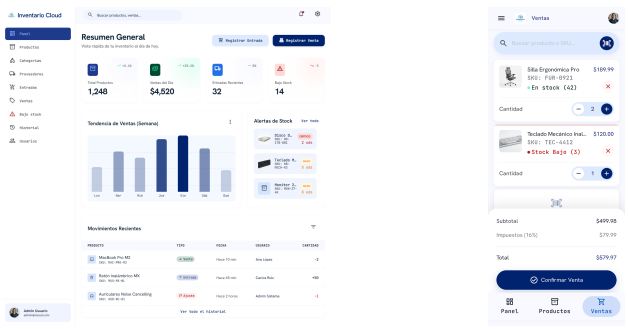



**Figura 3:** Modelo entidad-relación de la base de datos.

ajuste.

## 5.3 Servicios que ofrece el sistema

Para que el sistema sea fácil de usar, primero diseñamos sus pantallas. En la figura 4 se ven dos de ellas: el panel principal en la computadora y el registro de una venta en el celular. Los datos que aparecen son de ejemplo.



(a) Computadora (b) Celular

**Figura 4:** Pantallas del sistema.

Además, para las personas que trabajan en la tienda, el sistema ofrece los servicios del cuadro 3.

| Servicio | Descripción | Quién lo usa |
|---|---|---|
| Inicio de sesión | Entrar y salir con correo y contraseña. Cada usuario ve solo lo que le corresponde. | Todos |
| Usuarios y roles | Cambiar el rol de un usuario y activar o desactivar cuentas. | Administrador |
| Categorías | Registrar, editar y desactivar las categorías de productos. | Administrador |
| Proveedores | Registrar y consultar los proveedores con su teléfono, correo y dirección. | Administrador |
| Productos | Registrar productos con código, precio y stock mínimo. Buscar por nombre o código. | Administrador; el empleado solo consulta |
| Entradas de mercadería | Registrar una compra a un proveedor con uno o varios productos. El stock sube solo. | Administrador |
| Ventas | Armar un carrito, confirmar la venta y descontar el stock. Si no alcanza el stock, la venta no se guarda. | Todos |
| Ajustes de stock | Corregir el stock después de un conteo físico, indicando el motivo. | Administrador |
| Alertas de bajo stock | Ver los productos que llegaron a su stock mínimo o se agotaron. | Todos |
| Historial | Ver las entradas, ventas y ajustes con fecha, usuario y cantidad. | Administrador ve todo; el empleado, lo suyo |
| Respaldo | Exportar la base de datos para no perder la información. | Administrador |

**Cuadro 3:** Servicios que ofrece el sistema.

# Referencias

[1] Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I. y Zaharia, M. (2010). A view of cloud computing. *Communications of the ACM*, 53(4), 50–58.

[2] Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures*. Tesis doctoral, University of California, Irvine.

[3] Google. (s.f.). *Flutter documentation*. https://docs.flutter.dev

[4] Heizer, J. y Render, B. (2014). *Principios de administración de operaciones* (9.ª ed.). Pearson Educación.

[5] Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

[6] Mell, P. y Grance, T. (2011). *The NIST definition of cloud computing* (Special Publication 800-145). National Institute of Standards and Technology.

[7] PostgreSQL Global Development Group. (s.f.). *PostgreSQL documentation: Row security policies*. https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[8] Silberschatz, A., Korth, H. F. y Sudarshan, S. (2020). *Database system concepts* (7.ª ed.). McGraw-Hill.

[9] Supabase. (s.f.). *Supabase documentation*. https://supabase.com/docs

[10] Supabase. (2026). *Pricing*. Consultado el 21 de octubre de 2026. https://supabase.com/pricing